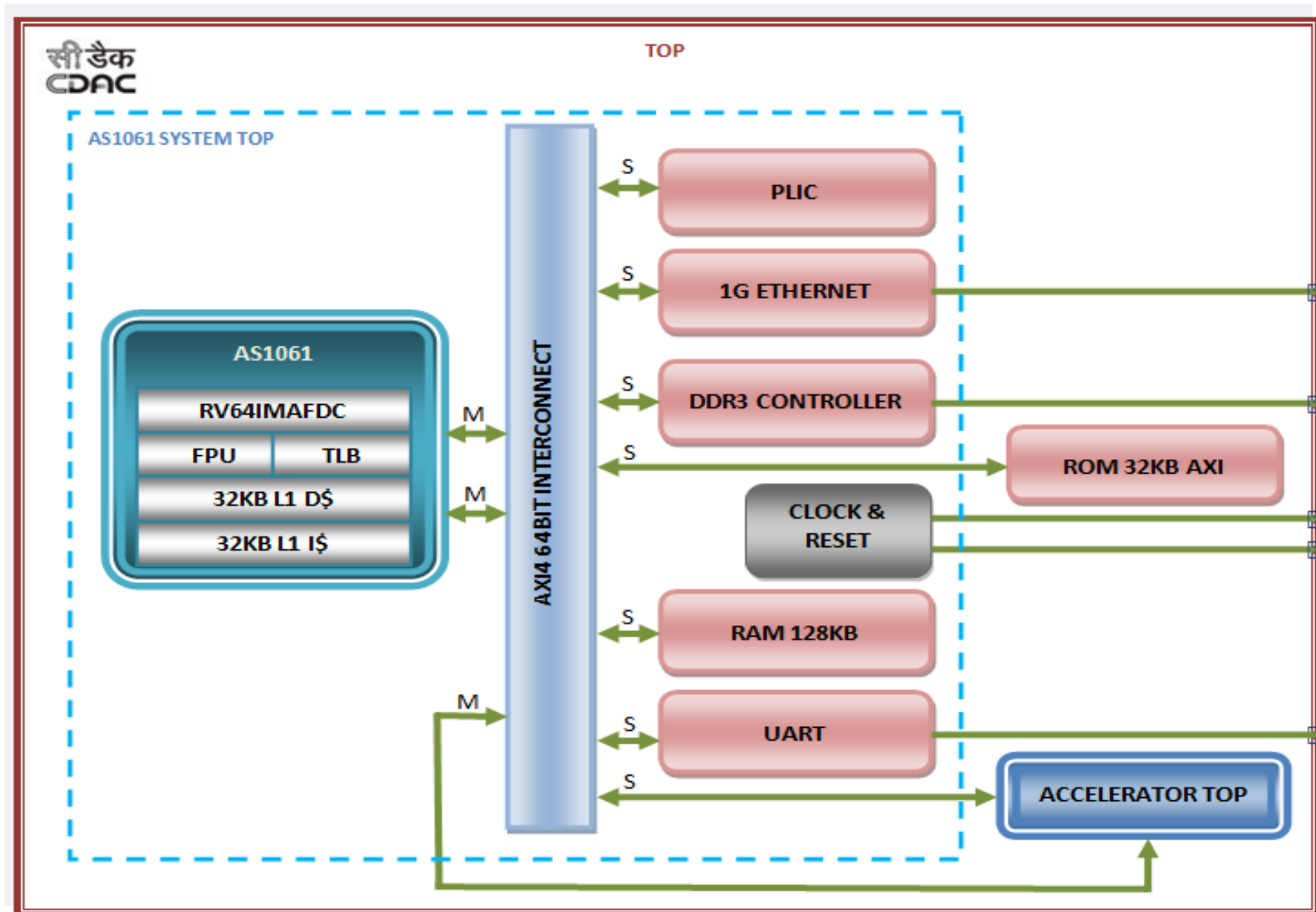


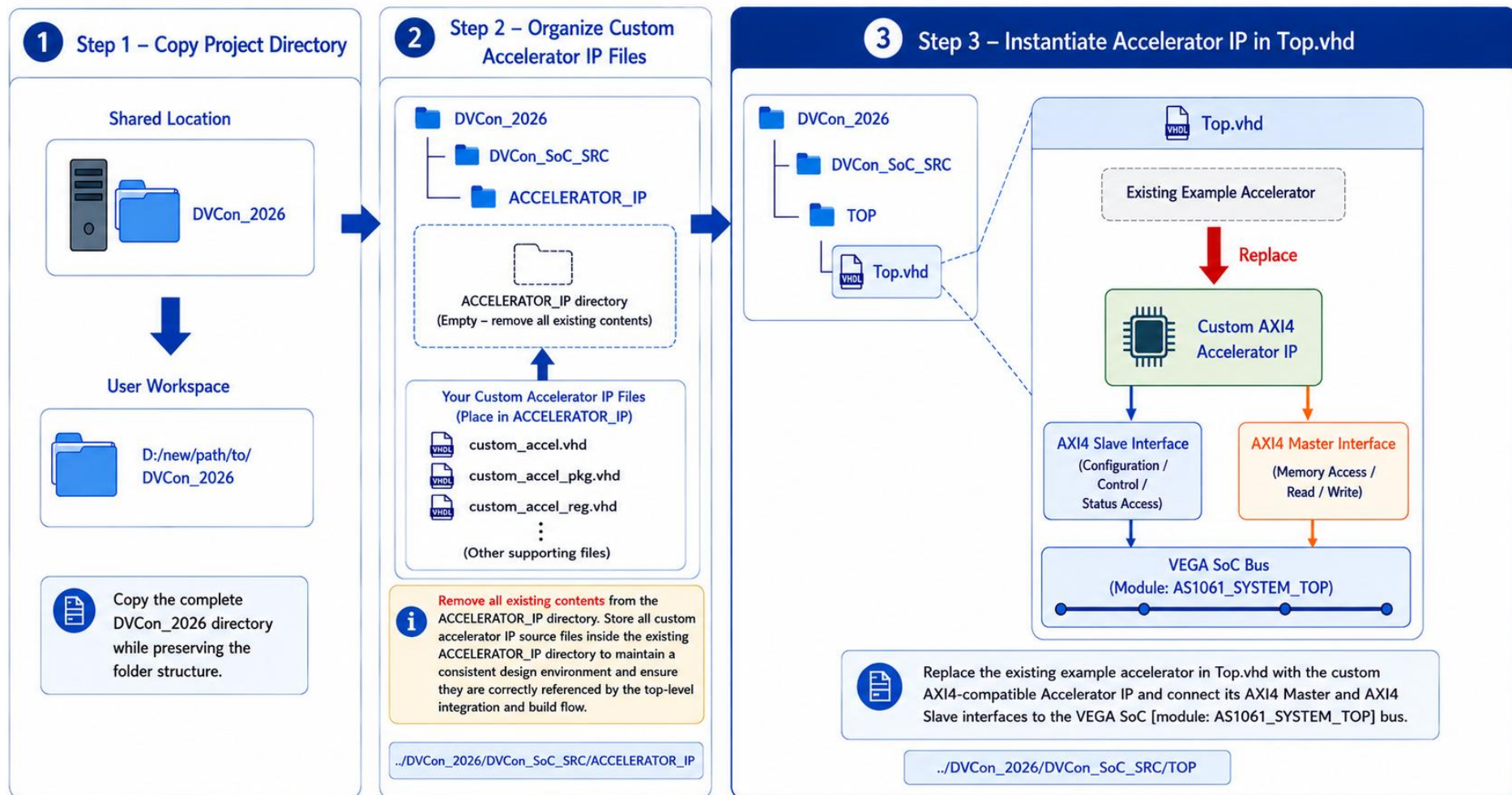
VEGA SoC for DVCON 2026



Guide to Simulation and Implementation of DVCON Project in Vivado

Integrating a custom Accelerator IP into the VEGA SoC requires updating the top-level design, replacing the example accelerator, and connecting the new IP to the required SoC interfaces. Follow the steps below to complete the integration, simulation, and implementation flow.

Guide to Integrating a custom Accelerator IP into the VEGA SoC



Guide to Use run.sh Script



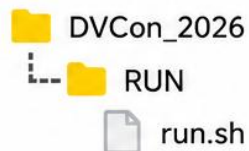
This script automates and streamlines your FPGA design workflow by presenting a simple terminal-based menu. It internally runs Vivado/QuartaSim TCL scripts for simulation, implementation, and FPGA programming.

1

Step 1: Execute the Script

- Run the script with:

```
./run.sh
```



```
user@linux:~/DVCon_2026/RUN
user@linux:~/DVCon_2026/RUN$ ./run.sh
```

2

Step 2: Choose an Option from the Menu

- Select the desired operation by entering the corresponding number.

```
user@linux:~/DVCon_2026/RUN
=====
                      FPGA Project Tool Menu
Main Directory:  /your/path/here
=====
1) Simulation
2) FPGA Implementation
3) FPGA Programming
4) Exit
=====
Enter choice [1-4]: █
```

3

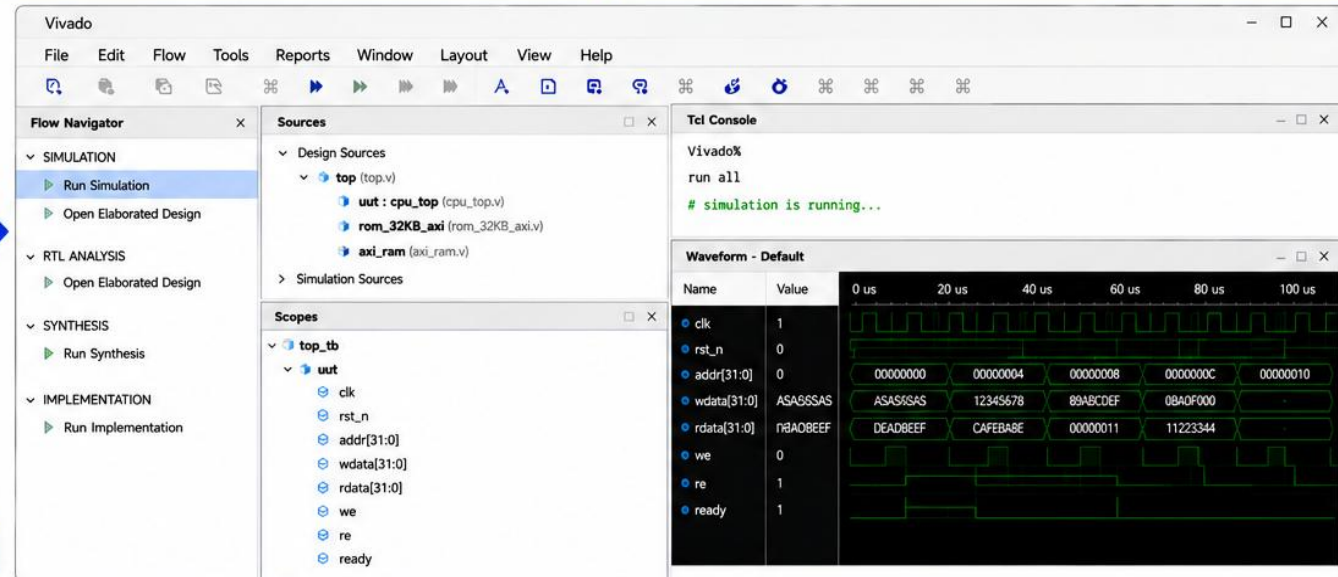
Step 3: Run Simulation (Option 1)

- Select the desired operation by entering the corresponding number.

```
user@linux:~/DVCon_2026/RUN
-----
                      Choose Simulation Type
-----
1) Vivado Simulation
2) QuestaSim Simulation
-----
Enter choice [1-2]: █
```


Vivado Simulation (Option 1)

```
user@linux:~/DVCon_2026/RUN - □ X
Running Vivado Simulation...
```

Flow Navigator

- SIMULATION
 - Run Simulation
 - Open Elaborated Design
- RTL ANALYSIS
 - Open Elaborated Design
- SYNTHESIS
 - Run Synthesis
- IMPLEMENTATION
 - Run Implementation

Sources

- Design Sources
 - top (top.v)
 - uut : cpu_top (cpu_top.v)
 - rom_32KB_axi (rom_32KB_axi.v)
 - axi_ram (axi_ram.v)
- Simulation Sources

Scopes

- top_tb
 - uut
 - clk
 - rst_n
 - addr[31:0]
 - wdata[31:0]
 - rdata[31:0]
 - we
 - re
 - ready

Tcl Console

```
Vivado%
run all
# simulation is running...
```

Waveform - Default

Name	Value	0 us	20 us	40 us	60 us	80 us	100 us
clk	1	[Waveform]					
rst_n	0	[Waveform]					
addr[31:0]	0	00000000	00000004	00000008	0000000C	00000010	
wdata[31:0]	ASASASAS	ASASASAS	12345678	89ABCDEF	0BA0F000		
rdata[31:0]	DEADBEEF	DEADBEEF	CAFEBA8E	00000011	11223344		
we	0	[Waveform]					
re	1	[Waveform]					
ready	1	[Waveform]					

Follow These Steps After Vivado Launches:

1 Monitor the Simulation:



- Monitor the Vivado Tcl Console to track the progress.
- Wait for Vivado to enter simulation mode and open the waveform viewer.

2 Update the .mif (Program image) File:



```
../DVCon_2026/
DVCon_SoC_SRC/
MEMORY_IP/
rom_32KB_axi/
```



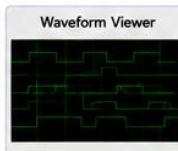
- Replace the existing rom_32KB_axi.mif file with your program image (same filename).
- After replacing the file, relaunch the simulation using the Relaunch Simulation button in the toolbar or Run → Relaunch Simulation in Vivado for the changes to take effect.

3 Add Signals to the Waveform Viewer:



- In Scopes or Objects, select signals and right-click "Add to Wave Window".

Right-click and Add to Wave Window



4 Run the Simulation:



run 1000 us

- Simulate for 1000 us.
- Modify duration as needed (e.g., run 500 us, run all).

5 View Simulation Results:

Name	Value	0 us	20 us	40 us	60 us	80 us	100 us
clk	1	[Waveform]					
rst_n	0	[Waveform]					
addr[31:0]	00000000	00000004	00000008	0000000C	00000010		
wdata[31:0]	ASASASAS	ASASASAS	12345678	89ABCDEF	0BA0F000		
rdata[31:0]	DEADBEEF	DEADBEEF	CAFEBA8E	00000011	11223344		
we	0	[Waveform]					
re	1	[Waveform]					
ready	1	[Waveform]					

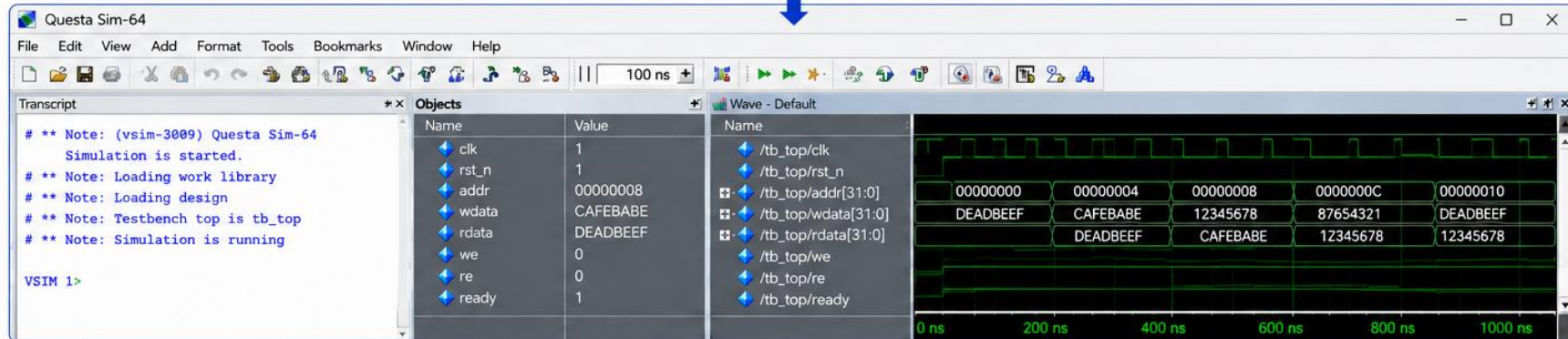
- Observe signal activity in the waveform viewer.
- Confirm whether the design functions as expected.



Verify correct design functionality.

QuestaSim Simulation (Option 2)

```
user@linux:~/DVCon_2026/RUN
Running QuestaSim Simulation...
```



1 Monitor Simulation



- Observe Transcript window
- Wait for testbench initialization

2 Add Signals



Objects /
Signals



Add to Wave



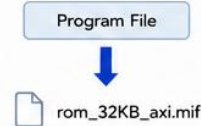
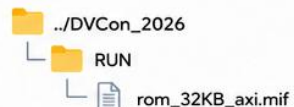
Selected
Signals



Wave Window

- Select the required signals and choose **Add to Wave → Selected Signals.**

3 Update the .mif (Program image) File



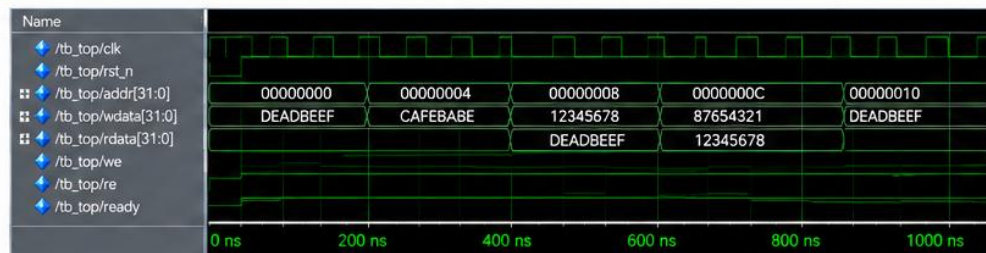
- Replace the existing rom_32KB_axi.mif file in this folder with your program image, ensuring that the filename remains **rom_32KB_axi.mif**.
- This file is used to pre-load the ROM during simulation.

4 Run Simulation

run 1000 us

- Simulation duration can be modified (run 500 us, run all, etc.)

5 View Results



Waveform
Window



Signal Activity



Verify correct
design functionality.

FPGA Implementation (Option 2) & FPGA Programming (Option 3)

1 Step 1: Choose an Option from the Menu



Options 2 and 3 will be provided in the next stage.

```
user@linux:~$ ./run.sh
=====
                        FPGA Project Tool Menu
                    Main Directory: /your/path/here
=====
1) Simulation
2) FPGA Implementation
3) FPGA Programming
4) Exit
=====
Enter choice [1-4]: █
```

EXIT the Script (Option 4)

1 Step 1: Choose an Option from the Menu

- Select **Option 4 – Exit** from the FPGA Project Tool Menu.
- Use this option to close the tool after simulation, implementation, or FPGA programming, or to exit without performing any operation.

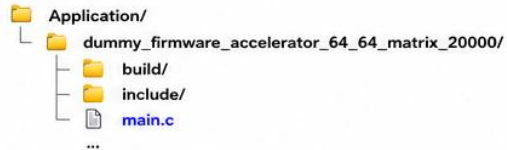
```
user@linux:~$ ./run.sh
=====
                        FPGA Project Tool Menu
                    Main Directory: /your/path/here
=====
1) Simulation
2) FPGA Implementation
3) FPGA Programming
4) Exit
=====
Enter choice [1-4]: █
```

Simulation of Bare-Metal Application



This workflow explains how to compile, customize, and simulate a bare-metal application for the VEGA SoC accelerator platform.

1 Modify the Application

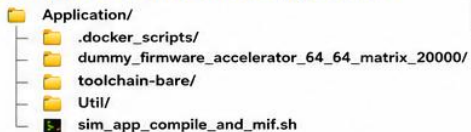


- Add or modify your application inside the main() function.
- Rebuild the application after making changes.

```
main.c
1 int main()
2 {
3     // User application
4     accelerator_test();
5     return 0;
6 }
7
```



2 Compile the Bare-metal Application



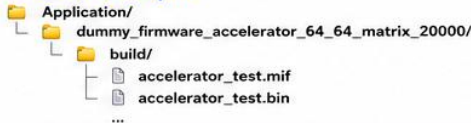
```
$ ./sim_app_compile_and_mif.sh
```

- Compile the firmware.
- Generate the executable image.
- Automatically create the ROM MIF file.

```
Terminal
Compiling application...
Generating ELF...
Generating MIF...
Build completed successfully.
```



3 Generated Output



- The generated MIF file is used during simulation.
- All compiled outputs are stored in the build folder.



4 Application Memory Layout

ROM	
Start Address	0x10000
Program Code	
SRAM	
Start Address	0x20000
Data	
Stack	
Heap	

- ROM Start Address : 0x10000
- SRAM Start Address : 0x20000

- ROM stores the program image.
- SRAM stores runtime data.
- Memory addresses are automatically configured.



5 Default Example Application



- 1 Matrix A and Matrix B are read from memory.
- 2 The hardware accelerator performs a 64 × 64 matrix multiplication.
- 3 Matrix C is written back to memory.



6 Docker Environment



```
Terminal
$ sudo usermod -aG docker <username>
```

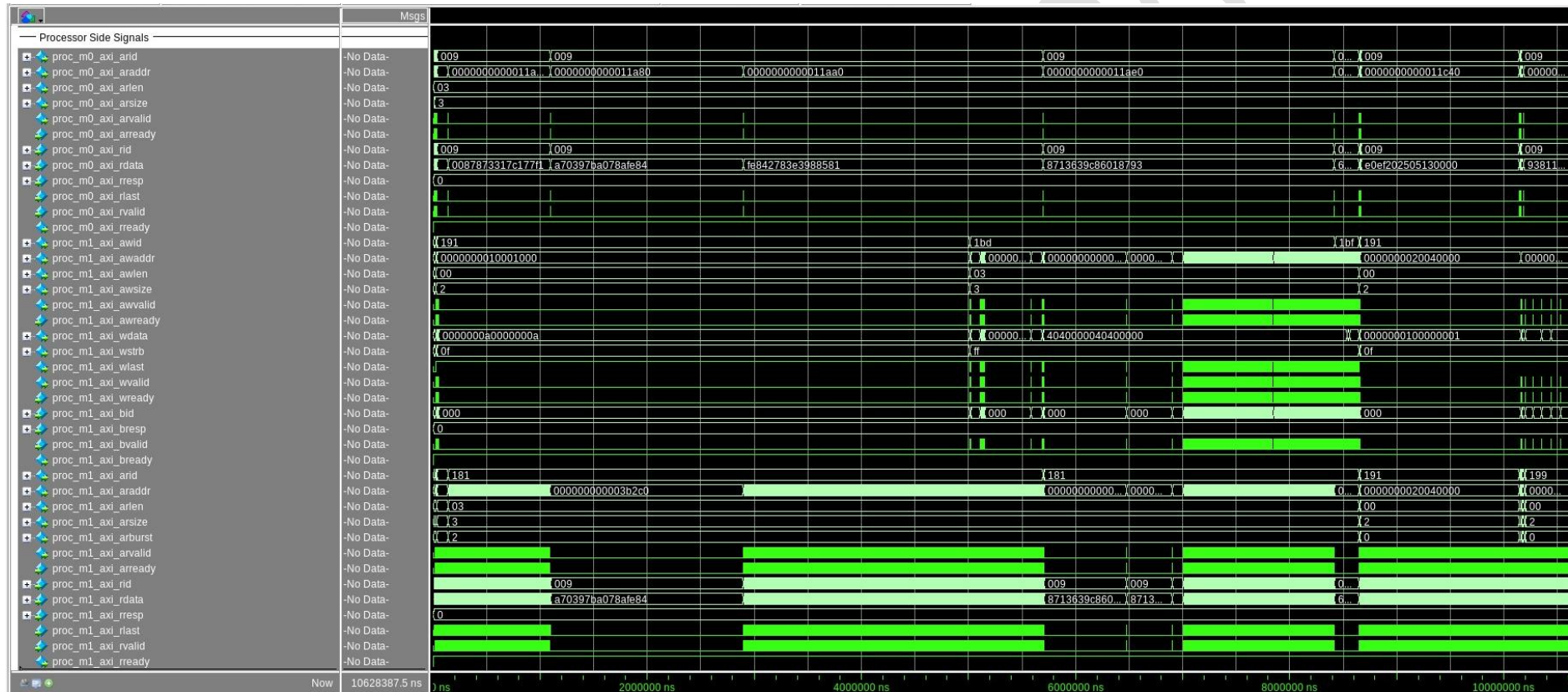


Important

- All application scripts are designed to run inside Docker.
- If Docker is not installed, the scripts automatically install Docker and load the provided image.
- Restart the system after adding the user to the Docker group.

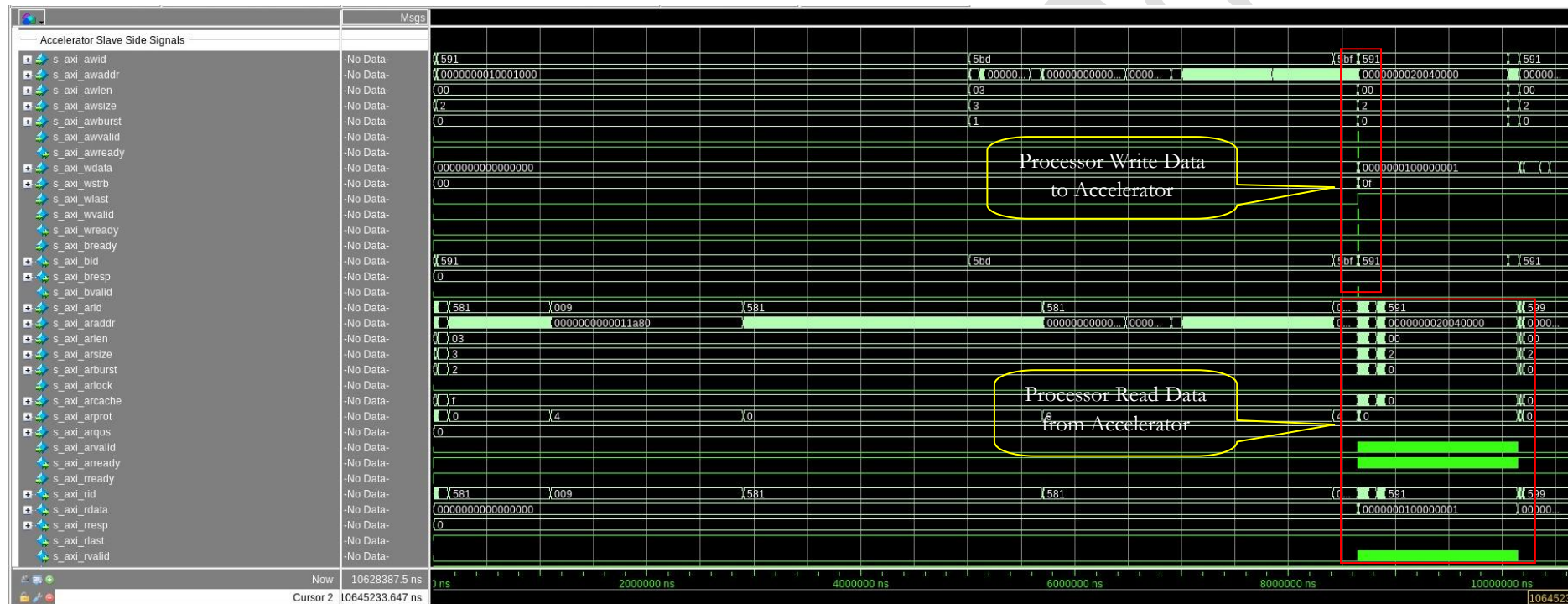
Example Accelerator Simulation Waveforms

Processor Interface Signals



Example Accelerator Simulation Waveforms

Accelerator Slave Side Interface Signals



Example Accelerator Simulation Waveforms

Accelerator Master Side Interface Signals

